

Reviewer Pack — Minimal Monodromy Group Order and Communication Complexity R...

Entry cf93be6cb223 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 01:06:10 UTC

Minimal Monodromy Group Order and Communication Complexity Rank

Entry ID: cf93be6cb223 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 01:06:10 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Monodromy Groups) **Field B** (complexity object): Communication Complexity

Statement:

For every Boolean function f with n variables, the order of its monodromy group M_f is lower-bounded by $\Omega(n^c \log n)$, where c is a constant, and this bound holds with respect to the communication complexity rank $r(f)$ such that $r(f) = \Theta(n^c \log n)$.

Rationale (proposer's reasoning):

Monodromy groups capture the algebraic symmetries of complex functions, which may reveal underlying structural properties related to computational tasks. A strong lower-bound on the monodromy group order could imply a deep connection between algebraic structure and communication complexity, suggesting that certain functions require significant communication for their computation.

Taxonomy category: MONODROMY_GROUP_ORDER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 07ad76139127026d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated Boolean functions f with n variables ($n \leq 40$), the order of the monodromy group M_f is at least $\Omega(n^c \log n)$ and the communication complexity rank $r(f)$ is within $\Theta(n^c \log n)$, based on at least 30 random seeds. The criterion is falsified if any seed results in an order of $M_f < \Omega(n^c \log n)$ or a rank $r(f)$ not within $\Theta(n^c \log n)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "monodromy group order" AND "communication complexity" AND Boolean function" - "algebraic geometry" AND "minimal monodromy group order" AND communication complexity" - order of monodromy group AND $\Omega(n^c \log n)$ AND communication complexity rank

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

# Helper functions for matrix operations
def matrix_multiply(A, B):
    m = len(A)
    n = len(B[0])
    p = len(B)
    result = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                result[i][j] += A[i][k] * B[k][j]
    return result

def gaussian_elimination(A, b):
    m = len(A)
    n = len(A[0])
    augmented_matrix = [A[i] + [b[i]] for i in range(m)]

    for i in range(n):
        # Find the pivot row
        max_row = i
        for j in range(i+1, m):
            if abs(augmented_matrix[j][i]) > abs(augmented_matrix[max_row][i]):
                max_row = j

        # Swap rows
        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]
```

```

    # Eliminate below the pivot
    for j in range(i+1, m):
        factor = augmented_matrix[j][i] / augmented_matrix[i][i]
        for k in range(n + 1):
            augmented_matrix[j][k] -= factor * augmented_matrix[i][k]

    # Back-substitute to find the solution
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = augmented_matrix[i][-1]
        for j in range(i+1, n):
            x[i] -= augmented_matrix[i][j] * x[j]
        x[i] /= augmented_matrix[i][i]

    return x

def determinant(A):
    m = len(A)
    if m == 1:
        return A[0][0]
    det = Fraction(0)
    for j in range(m):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        det += (-1)**j * A[0][j] * determinant(submatrix)
    return det

def monodromy_group_order(f):
    n = len(f)
    if n == 1:
        return 2

    # Generate all possible assignments
    assignments = [list(range(2)) for _ in range(n)]
    all_assignments = list(itertools.product(*assignments))

    # Find the monodromy group elements
    monodromy_elements = set()
    for perm in itertools.permutations(range(n)):
        new_f = []
        for assignment in all_assignments:
            new_assignment = [assignment[perm[i]] for i in range(n)]
            new_f.append(new_assignment)
        if new_f == f:
            monodromy_elements.add(tuple(perm))

    # The order of the monodromy group is the number of unique permutations
    return len(monodromy_elements)

def communication_complexity_rank(f):
    n = len(f)
    if n == 1:
        return 1

    # Generate all possible assignments
    assignments = [list(range(2)) for _ in range(n)]

```

```

all_assignments = list(itertools.product(*assignments))

# Find the rank of the communication complexity matrix
M = []
for assignment in all_assignments:
    row = []
    for i in range(n):
        row.append(assignment[i])
    M.append(row)

det = determinant(M)
if det == 0:
    return n

return n - 1

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_max = 40
    instances_tested = 30
    c = Fraction(1, 2) # Example constant for the bound

    results = []
    for _ in range(instances_tested):
        n = random.randint(5, 40)
        f = [[random.choice([0, 1]) for _ in range(n)] for _ in range(2**n)]

        order = monodromy_group_order(f)
        rank = communication_complex_rank(f)

        lower_bound = c * n**c * math.log(n)
        upper_bound = c * n**c * math.log(n)

        results.append({
            "order": order,
            "rank": rank,
            "lower_bound": lower_bound,
            "upper_bound": upper_bound
        })

    total_order = sum(result["order"] for result in results)
    total_rank = sum(result["rank"] for result in results)
    mean_order = total_order / instances_tested
    mean_rank = total_rank / instances_tested

    conjecture_holds = all(
        result["order"] >= result["lower_bound"] and result["rank"] <= result["upper_bound"]
        for result in results
    )

    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "monodromy_group_order",
        "metric_value": mean_order,
        "instances_tested": instances_tested,

```

```

        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\\"seed\\": {seed}, **{trial_result}}}")
        results.append(trial_result)

    mean_order = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_order} std=0.0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete within the allotted time frame. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	20473
2	propose	ollama_remote	glm4:latest	0	0	14426
3	propose	ollama_remote	glm4:latest	0	0	24951
4	preregistration	ollama_remote	glm4:latest	0	0	17665
5	novelty	ollama_remote	glm4:latest	0	0	11141
6	novelty	ollama_remote	glm4:latest	0	0	17439
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13618
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22537
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	68100
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20690
11	judge	ollama_remote	glm4:latest	0	0	39172

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 270212 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in *research/audit/cf93be6cb223.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/cf93be6cb223.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/cf93be6cb223.tar.gz* (if generated)